Question Bank — Week 7

Queues (Lecture 07) — Questions

DSA27 · Data Structures and Algorithms

Fall 2026

Contents

Part A — Multiple choice **2**

Part B — Short answer and essay **8**

Part C — Trace the code **9**

Part D — Ring state **10**

Part E — Complexity analysis **10**

Part F — Find and fix the bug **11**

Part G — Write the code **11**

> ***Answers are in a separate file:*** *`week07-answers.md`. Levels:* ***[what]*** *recall ·* ***[how]*** *apply ·* ***[why]*** *explain and justify. `SlowQueue` and `CircularQueue` are the classes of `dsa/queue.py`: `enqueue`, `dequeue` (raising `IndexError` when empty) and `len`; `CircularQueue` also has `is_empty`, `is_full` and iteration front to back. A `CircularQueue` stores its items in a fixed `Array` with `head`, `size` and `capacity`, and the next enqueue writes to slot `(head + size) % capacity`. `Stack` is Week 6's.*

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W7-M01 – W7-M22
B	Short answer and essay	W7-E1 – W7-E5
C	Trace the code	W7-T1 – W7-T5
D	Ring state — draw every step	W7-S1 – W7-S3
E	Complexity analysis	W7-K1 – W7-K3
F	Find and fix the bug	W7-B1 – W7-B4
G	Write the code — checked by `pytest`	W7-C1 – W7-C5

Part A — Multiple choice

W7-M01 [what] "FIFO" means:

- **a)** the last item added is the first one removed
- **b)** the first item added is the first one removed
- **c)** items are removed in sorted order
- **d)** any item can be removed

W7-M02 [how] `enqueue(1)`, `enqueue(2)`, `enqueue(3)`, `dequeue()`, `enqueue(4)`, `dequeue()`. What does the **next** `dequeue()` return?

- **a)** 4
- **b)** 1
- **c)** 2
- **d)** 3

W7-M03 [why] In `SlowQueue`, `dequeue` is `pop(0)` on a `DynamicArray`. Its cost on a queue of n items is:

- **a)** $O(n)$ — every remaining element shifts one slot left
- **b)** $O(1)$
- **c)** amortised $O(1)$
- **d)** $O(\log n)$

W7-M04 [why] An array-based queue that enqueues with `insert_at(0, x)` and dequeues with `pop()` at the end has costs:

- **a)** both $O(1)$
- **b)** enqueue $O(n)$, dequeue $O(1)$
- **c)** enqueue $O(1)$, dequeue $O(n)$
- **d)** both $O(n)$

W7-M05 [how] A `CircularQueue` with capacity 8 has head = 6 and size = 3. The next `enqueue` writes to slot:

- **a)** 9
- **b)** 0
- **c)** 1
- **d)** 7

W7-M06 [how] A `CircularQueue` with capacity 5 has head = 4 and size = 2. After one `dequeue`, head is:

- **a)** 5
- **b)** 3
- **c)** 1
- **d)** 0

W7-M07 [what] A ring buffer that stores only `head` and `tail` (no size) finds `head == tail`. The queue is:

- **a)** full
- **b)** empty
- **c)** either full or empty — it cannot tell which
- **d)** corrupted

W7-M08 [why] `CircularQueue` keeps a `_size` field because:

- **a)** it tells a full ring from an empty one, which look the same by head and tail
- **b)** `len` would otherwise be $O(n)$ with no other benefit
- **c)** the `Array` does not know its own length
- **d)** Python requires it for `__iter__`

W7-M09 [how] A full ring of capacity 4 holds `[E, F, C, D]` with head = 2. It grows to capacity 8 by copying `new[i] = old[i]` and keeping head = 2. Read front to back, the queue is now:

- **a)** C D E F
- **b)** C D None None
- **c)** E F C D
- **d)** D C F E

W7-M10 [what] A queue on a singly linked list with `head` and `tail` references should put the front of the queue at:

- **a)** the tail
- **b)** the middle node
- **c)** either end — it makes no difference
- **d)** the head

W7-M11 [why] Why not dequeue at the **tail** of a singly linked list?

- **a)** Removing the last node needs the node before it, which takes an $O(n)$ walk
- **b)** The tail node has no value
- **c)** It would make the queue LIFO
- **d)** Python forbids it

W7-M12 [how] A queue built from two stacks (`inbox` and `outbox`, pouring only when `outbox` is empty): `enqueue(1)`, `enqueue(2)`, `enqueue(3)`, `dequeue()`. How many items are now in `outbox`?

- **a)** 0
- **b)** 1
- **c)** 2
- **d)** 3

W7-M13 [why] The cost of `dequeue` on that two-stack queue is:

- **a)** $O(n)$ on average
- **b)** $O(\log n)$
- **c)** $O(1)$ in the worst case
- **d)** amortised $O(1)$ — each item is poured from `inbox` to `outbox` at most once

W7-M14 [what] Which of these needs a **queue**?

- **a)** checking balanced brackets
- **b)** undo in an editor
- **c)** breadth-first search of a graph
- **d)** evaluating a postfix expression

W7-M15 [how] n enqueues followed by n dequeues on a `SlowQueue` cost in total:

- **a)** $\Theta(n^2)$
- **b)** $\Theta(n)$
- **c)** $\Theta(n \log n)$
- **d)** $\Theta(1)$

W7-M16 [how] In Python, `(0 - 1) % 8` is:

- **a)** −1
- **b)** 7
- **c)** 0
- **d)** an error

W7-M17 [what] A **deque** is a structure that:

- **a)** serves the most urgent item first
- **b)** can only be used as a stack
- **c)** removes items in sorted order
- **d)** adds and removes at both ends

W7-M18 [why] Measured, a ring-buffer dequeue and a linked-queue dequeue are both flat lines as n grows, but the ring is about five times slower. The reason:

- **a)** a constant factor: the `%` and the bounds-checked `Array` access on every operation
- **b)** the ring is really $O(\log n)$
- **c)** the ring copies the queue on every dequeue
- **d)** the linked queue is $O(1)$ and the ring is $O(n)$

W7-M19 [how] Five people A, B, C, D, E stand in a circle; counting from A, every **second** person leaves. Who is left last?

- **a)** A
- **b)** E
- **c)** C
- **d)** D

W7-M20 [how] An array queue with a head index but **no** wrap-around, 8 slots: 8 enqueues, then 5 dequeues. How many more enqueues fit without growing the block?

- **a)** 5
- **b)** 0
- **c)** 3
- **d)** 8

W7-M21 [why] A `CircularQueue` that doubles its capacity when full has an enqueue cost of:

- **a)** $O(n)$ every time

- **b)** $O(\log n)$
- **c)** $O(1)$ in the worst case
- **d)** amortised $O(1)$

W7-M22 [what] A **priority queue** (Week 12) differs from a queue because it dequeues:

- **a)** the most urgent item, not the oldest
- **b)** the newest item
- **c)** a random item
- **d)** from both ends

W7-M23 [how] A `CircularQueue(4)`, starting empty: `enqueue(1)`, `enqueue(2)`, `enqueue(3)`, `enqueue(4)`, `dequeue()`, `dequeue()`, `enqueue(5)`. Which describes the ring now?

- **a)** head = 0, size = 3, and 5 is in slot 0
- **b)** head = 2, size = 3, and 5 is in slot 4
- **c)** head = 2, size = 3, and 5 is in slot 0
- **d)** head = 2, size = 4, and 5 is in slot 1

W7-M24 [how] A `CircularQueue(5)`, starting empty: enqueue A, B, C, D, E; dequeue three times; enqueue F, then G. What is in the block, from slot 0 to slot 4?

- **a)** `F G None D E`
- **b)** `D E F G None`
- **c)** `None None None D E F G`
- **d)** `F G C D E`

W7-M25 [how] A full `CircularQueue` of capacity 4 holds C D E F (front to back), with head = 2. `enqueue(G)` makes the correct `_grow` double the block first. Afterwards, head is ___ and G is in slot ___:

- **a)** 2 and 6
- **b)** 0 and 0
- **c)** 2 and 2
- **d)** 0 and 4

W7-M26 [why] A student's `_grow` copies in queue order into an `Array(8)` and sets `head = 0`, but forgets to set `self._capacity = 8`. On the full ring of W7-M25, `enqueue(G)` then:

- **a)** works, because the new block really has 8 slots
- **b)** writes G to slot `(0 + 4) % 4 = 0`, over the front item C
- **c)** raises `IndexError`, because slot 4 is out of range
- **d)** writes G to slot 4, but `len(q)` stays 4

W7-M27 [why] A `dequeue` updates the head with `self._head += 1` instead of `self._head = (self._head + 1) % self._capacity`. On a `CircularQueue(4)`: enqueue A, B, C, D; dequeue four times; enqueue E; dequeue. What happens?

- **a)** The first dequeue fails: head starts beyond the block
- **b)** Every operation works; the last dequeue returns E
- **c)** The last dequeue raises `IndexError`: it reads slot 4 of a 4-slot block
- **d)** The last dequeue returns `None`

W7-M28 [why] A linked queue's `enqueue` tests `if self._tail is None:` to decide that the queue is empty, and its `dequeue` moves `head` on but **never** resets `tail`. After `enqueue(1)`, `dequeue()`, `enqueue(2)`, the next `dequeue()`:

- **a)** raises `IndexError`: 2 was linked after the old, removed node, and `head` is still `None`
- **b)** returns 2
- **c)** returns 1 again
- **d)** loops for ever

W7-M29 [how] A queue from two stacks (pour only when `outbox` is empty). Count every `push` and pop on the two stacks, including the pours: enqueue 1, 2, 3; dequeue; enqueue 4, 5; dequeue three times. How many stack operations is that in total?

- **a)** 13
- **b)** 15
- **c)** 25
- **d)** 19

W7-M30 [what] A ring buffer that **leaves one slot unused** to tell full from empty has a block of 8 slots. It holds at most:

- **a)** 8 items
- **b)** 7 items
- **c)** 9 items
- **d)** 4 items

W7-M31 [how] A ring with 8 slots stores only `head` and `tail` and leaves one slot unused. Now head = 3 and tail = 2. How many items does it hold, and is it full?

- **a)** 1 item, not full
- **b)** 0 items: `head` and `tail` are next to each other, so it is empty
- **c)** 7 items, full: `(tail + 1) % 8 == head`
- **d)** 8 items, full

W7-M32 [why] `CircularQueue.dequeue` sets the slot it read back to `None`. Why?

- **a)** `is_empty` checks for `None` in the head slot
- **b)** so the block no longer refers to the dequeued object, which can then be freed
- **c)** `enqueue` looks for a `None` slot to write into
- **d)** without it, `__iter__` would show the dequeued value

W7-M33 [what] Which of these is a typical use of a **ring buffer**?

- **a)** checking that brackets are balanced
- **b)** the undo history of an editor
- **c)** sorting a list of names
- **d)** keeping the last 1,000 lines of a log, overwriting the oldest

W7-M34 [how] What does this print?

```
q = CircularQueue()
q.enqueue("1")
```

```
for _ in range(6):
    s = q.dequeue()
    print(s, end=" ")
    q.enqueue(s + "0")
    q.enqueue(s + "1")
```

- **a)** `1 10 11 100 101 110`
- **b)** `1 10 100 1000 10000 100000`
- **c)** `1 11 10 111 110 101`
- **d)** `1 10 11 101 100 110`

W7-M35 [how] Six people, 1 to 6, are put in a queue. Until one is left: move two people from the front to the back, then dequeue the third, who leaves. In what order do they leave, and who is left?

- **a)** 3, 6, 2, 5, 4 — and 1 is left
- **b)** 1, 4, 2, 6, 3 — and 5 is left
- **c)** 3, 6, 4, 2, 5 — and 1 is left
- **d)** 3, 4, 5, 6, 1 — and 2 is left

W7-M36 [what] A graph search dequeues a node, then enqueues its unvisited neighbours, and repeats. If the queue is replaced by a **stack** (pop instead of dequeue, push instead of enqueue), the same loop:

- **a)** still visits the nodes breadth-first
- **b)** visits the nodes depth-first
- **c)** never stops
- **d)** visits only the start node

W7-M37 [what] With Python's `collections.deque`, the $O(1)$ way to remove the **front** item of a queue `d` is:

- **a)** `d.pop(0)`
- **b)** `d.pop()`
- **c)** `d.dequeue()`
- **d)** `d.popleft()`

W7-M38 [why] A moving average of the last `k` readings keeps a `CircularQueue(k)` **and** a running total beside it. The running total is there because:

- **a)** the ring cannot be iterated
- **b)** it tells a full window from an empty one
- **c)** each new average then costs $O(1)$, instead of $O(k)$ to add up the window again
- **d)** floating-point sums are only exact that way

W7-M39 [how] A `CircularQueue` with capacity 6 has head = 4 and size = 5. Front to back, the queue's items are in slots:

- **a)** 4, 5, 0, 1, 2
- **b)** 4, 5, 6, 7, 8
- **c)** 0, 1, 2, 3, 4

- **d)** 4, 3, 2, 1, 0

W7-M40 [why] A naive `_grow` copies `new[i] = old[i]` and keeps head. It passes a test that fills a fresh `CircularQueue(4)` with A, B, C, D and then enqueues E. Why does the test not catch the bug?

- **a)** The naive copy is in fact correct
- **b)** The ring has not wrapped: head is 0, so slot order and queue order are the same
- **c)** The test only checks `len(q)`
- **d)** `Array` reorders its slots when it grows

Part B — Short answer and essay

W7-E1 [why] *(4 marks)* State the Queue ADT with the cost of each operation. Explain why an array-based queue that shifts is $O(n)$ per dequeue, and why swapping which end is the front does not help. How does a ring buffer fix it?

W7-E2 [how] *(4 marks)* In a ring buffer, `head == tail` can mean two different things. Explain which two, with an example of each, and describe two ways of telling them apart. Which one does `dsa/queue.py` use?

W7-E3 [why] *(4 marks)* A full `CircularQueue` must grow. Explain, with an example, why copying the block slot by slot gives a wrong queue once the data has wrapped, give the correct copy, and explain why doubling keeps enqueue amortised $O(1)$.

W7-E4 [why] *(3 marks)* Describe a queue built from two stacks. Show that it is amortised $O(1)$ per operation, and explain why pouring must happen only when the `outbox` is empty.

W7-E5 [why] *(3 marks)* Compare the ring buffer and the linked queue (with head and tail): cost per operation, memory, and when you would choose each. Where does Python's `collections.deque` fit?

W7-E6 [why] *(4 marks)* Three ring-buffer bugs: the tail computed as `head + size` with no `%`; the head moved with `head += 1` with no `%`; and a `_grow` that never updates `_capacity`. For each, say what goes wrong and the shortest operation sequence that shows it. Why do all three pass a test that never wraps the ring?

W7-E7 [why] *(3 marks)* When a ring buffer is full, it can **refuse** (a bounded queue) or **grow**. Explain both choices, give a real use for each, and say which `level_order` and `bfs` need, and why.

W7-E8 [how] *(4 marks)* Describe `enqueue` and `dequeue` for a queue on a singly linked list with `head` and `tail`, including the empty cases. Explain why the front must be the head, and what goes wrong if `dequeue` does not reset `tail` when the queue becomes empty.

W7-E9 [why] *(3 marks)* Explain why breadth-first order needs a **queue**, using the binary numbers 1, 10, 11, 100, … generated from "1" (dequeue s, enqueue s + "0" and s + "1"). What order do you get if the queue is replaced by a stack, and why?

Part C — Trace the code

W7-T1 [how] What is printed? Then say which slot of the block holds each value at the end.

```
q = CircularQueue(capacity=4)
out = []
for v in (1, 2, 3):
    q.enqueue(v)
out.append(q.dequeue())
q.enqueue(4)
q.enqueue(5)
out.append(q.dequeue())
q.enqueue(6)
out.append(q.is_full())
out.append(list(q))
print(out)
```

W7-T2 [how] A two-stack queue (W7-C1) runs: `enqueue(1)`, `enqueue(2)`, `enqueue(3)`, `dequeue()`, `enqueue(4)`, `dequeue()`, `dequeue()`, `enqueue(5)`, `dequeue()`, `dequeue()`. Show both stacks (bottom to top) after every operation, and what each dequeue returns.

W7-T3 [how] Six people A–F stand in a circle; counting from A, every **third** person leaves (W7-C2). Show the queue, front to back, after each person leaves. In what order do they leave, and who is last?

W7-T4 [how] `binary_numbers(6)` (W7-C5) starts with the queue `["1"]`. Show the queue after each dequeue (and its two enqueues). What is returned?

W7-T5 [why] What is printed? Why are the two halves of the output in different orders?

```
def mystery(values):
    q, s = CircularQueue(len(values)), Stack()
    for v in values:
        q.enqueue(v)
    while len(q) > 0:
        v = q.dequeue()
        if v % 2 == 0:
            s.push(v)
        else:
            print(v, end=" ")
    while not s.is_empty():
        print(s.pop(), end=" ")

mystery([1, 2, 3, 4, 5, 6])
```

Part D — Ring state

W7-S1 [how] Draw the block, `head` and `size` after each operation on `q = CircularQueue(capacity=5)`. What is `list(q)` at the end, and is the queue full?

```
enqueue 10, enqueue 20, enqueue 30, enqueue 40, dequeue, dequeue,
enqueue 50, enqueue 60, enqueue 70, dequeue
```

W7-S2 [why] A student computes the tail as `self._head + self._size`, without `% self._capacity`. Draw what happens in `test_circular_queue_wraps_around`: capacity 3, enqueue 1, 2, 3, dequeue, enqueue 4. Why did the tests of earlier operations not catch it?

W7-S3 [how] A deque on a ring buffer, like `CircularQueue`, keeps `head` and `size`. `add_front` first steps `head` back one slot, `(head - 1) % capacity`, then writes there; `add_back` writes at `(head + size) % capacity`; `remove_back` takes the slot `(head + size - 1) % capacity`. Starting from an empty deque of capacity 4, draw the block, `head` and `size` after each operation, and the deque front to back:

```
add_back 1, add_back 2, add_front 0, add_front 9, remove_back,
add_back 5, remove_front, add_front 7, add_back 8
```

The last operation finds the deque full: it doubles the capacity first.

Part E — Complexity analysis

W7-K1 [how] Give Θ and justify:

```python
q = SlowQueue()
for i in range(n):
    q.enqueue(i)
while len(q) > 0:
    q.dequeue()
```

What is it with a `CircularQueue(n)` instead?

W7-K2 [how] Give Θ and justify:

```python
q = CircularQueue(n)
for i in range(n):
    q.enqueue(i)
    print(sum(q))                # sum iterates over the whole queue
```

W7-K3 [why] A student's two-stack queue pours `inbox` into `outbox` on **every** dequeue, and pours it back afterwards. Give the cost of n enqueues followed by n dequeues, and compare with the correct version.

Part F — Find and fix the bug

W7-B1 [how] `SlowQueue`:

```
def dequeue(self):
    if len(self._items) == 0:
        raise IndexError("dequeue from empty queue")
    return self._items.pop()
```

W7-B2 [why] `CircularQueue` — this passes `test_circular_queue_wraps_around`:

```
def dequeue(self):
    if self._size == 0:
        raise IndexError("dequeue from empty queue")
    value = self._block[self._head]
    self._block[self._head] = None
    self._head += 1
    self._size -= 1
    return value
```

W7-B3 [how] `CircularQueue`:

```
def enqueue(self, value):
    tail = (self._head + self._size) % self._capacity
    if tail == self._head:
        raise IndexError("queue is full")
    self._block[tail] = value
    self._size += 1
```

W7-B4 [why] A linked queue with `head` and `tail`:

```
def dequeue(self):
    if self.head is None:
        raise IndexError("dequeue from empty queue")
    value = self.head.value
    self.head = self.head.next
    return value
```

Part G — Write the code

In `practice/week07.py`; check with `pytest tests/test_practice_week07.py -v`. Use your own `CircularQueue`, `Stack` or the course `Array` as working storage.

W7-C1 [why] `StackQueue` — a queue from two stacks. Why is it amortised $O(1)$?

W7-C2 [how] `josephus(names, k)` — the order in which people leave the circle.

W7-C3 [why] `moving_averages(values, k)` — the average of every window of k values, with

a `CircularQueue` as the window. Why is it $O(n)$ and not $O(nk)$?

W7-C4 [how] `reverse_first_k(values, k)` — with one queue and one stack.

W7-C5 [how] `binary_numbers(n)` — the binary strings 1 to n, generated breadth-first.